
SQL Letal: El Misterio de la Base de Datos

Una Novela de Misterio, Datos y Consultas

Alex Goyzueta Delgado

alexgoyzueta2018@gmail.com

SQL Letal: El Misterio de la Base de Datos

© 2026 Alex Goyzueta Delgado

Todos los derechos reservados. Ninguna parte de esta publicación, incluido el diseño de la cubierta, puede ser reproducida, almacenada o transmitida de ninguna forma ni por ningún medio, sea electrónico, químico, mecánico, óptico, de grabación o fotocopia, sin permiso previo del autor.

Contacto: alexgoyzueta2018@gmail.com

Edición: Alex Goyzueta Delgado

Corrección técnica: El autor

Diseño de portada: Alex Goyzueta Delgado

Inspiración: Los casos reales donde SQL resolvió crímenes financieros, fraudes de datos y misterios empresariales.

Los personajes y eventos descritos en esta obra son ficticios. Cualquier semejanza con personas reales, vivas o fallecidas, o con hechos reales, es pura coincidencia.

Las consultas SQL y ejemplos de código incluidos en este libro son funcionales y pueden

ejecutarse en SQLite, PostgreSQL y la mayoría de los motores de bases de datos relacionales con cambios mínimos.

Goyzueta Delgado, Alex

SQL Letal: El Misterio de la Base de Datos / Alex Goyzueta Delgado

1ra edición — Barcelona, 2026

Dedicatoria

A los administradores de bases de datos que trabajan en la sombra, velando por la integridad de los datos que mueven el mundo.

A los analistas que saben que la respuesta correcta está en la consulta adecuada.

Y a todos los que alguna vez escribieron ``SELECT * FROM vida WHERE esperanza = true``.

Prefacio

SQL es el lenguaje más importante que quizás no sabes que necesitas. Desde las aplicaciones que usamos a diario hasta los sistemas financieros globales, todo funciona sobre bases de datos. Y SQL es la llave que abre esas puertas.

Este libro no es un manual técnico tradicional. Es una novela de misterio donde cada capítulo te enseña un concepto de SQL a través de una historia de crimen y descubrimiento. Acompañarás a Elisa Riera, una analista de datos, mientras usa consultas SQL para resolver el asesinato de su mentor.

No necesitas saber SQL para leer este libro. Los conceptos se introducen gradualmente, desde lo más básico (``SELECT``) hasta lo más avanzado (window functions, CTEs). Cada consulta está explicada en detalle.

Al final de cada capítulo encontrarás **enigmas SQL** para resolver. Las soluciones están en el Apéndice.

La historia está ambientada en Barcelona, 2026, una ciudad donde los datos fluyen por cada fibra óptica y donde la información es el recurso más valioso.

— Alex Goyzueta Delgado

Barcelona, 2026

Introducción

Barcelona, 2026

Imagina una ciudad donde cada transacción bancaria, cada llamada telefónica, cada correo electrónico, cada movimiento de inventario, cada historial médico queda registrado en una base de datos.

Imagina que alguien ha manipulado esos datos para ocultar un crimen.

Imagina que solo una persona puede desentrañar la verdad, siguiendo el rastro de las consultas.

Esa persona es Elisa Riera.

Elisa es analista de datos en el Instituto de Ciberseguridad de Barcelona (ICB). Su mentor, el Dr. Héctor Blasco, apareció muerto en su laboratorio una madrugada de julio. La policía dice que fue un ataque al corazón. Pero los datos cuentan otra historia.

Para descubrir la verdad, Elisa deberá sumergirse en bases de datos, seguir joins, filtrar pistas, agrupar sospechosos y ejecutar la consulta más importante de su vida.

Bienvenido a **SQL Letal**.

Al final de este viaje, no solo sabrás SQL. Sabrás cómo piensa un investigador de datos.

Capítulo 1: SELECT y los Datos Perdidos

Eran las 6:47 de la mañana cuando el teléfono de Elisa Riera vibró sobre la mesa de noche. El sonido era diferente al de una llamada normal: más grave, más insistente. Era el tono de emergencia del ICB.

Veinte minutos después estaba frente a la puerta del laboratorio 7, en el sótano del instituto. El lugar olía a café quemado y a plástico caliente. Un cuerpo yacía en el suelo, junto a una silla volcada. El Dr. Héctor Blasco, su mentor, tenía los ojos abiertos mirando el techo.

—Elisa, lo siento —dijo el inspector Rovira, acercándose—. Lo encontró el guardia de seguridad a las 5:30.

—¿Qué dice la autopsia?

—Ataque al corazón. Pero hay algo raro. Héctor estaba sano. Corría maratones.

Elisa se arrodilló junto al cuerpo. Sus dedos tocaron el brazo frío del profesor. Luego miró la pantalla de su ordenador. Estaba encendida, mostrando una ventana de terminal con una línea de texto:

```
SELECT * FROM experimentos;
```

—No ha ejecutado la consulta —murmuró Elisa—. Es una instrucción incompleta. Héctor nunca dejaba consultas a medias.

Concepto SQL: SELECT y FROM

La consulta ``SELECT * FROM experimentos;`` es la instrucción más fundamental en SQL. Vamos a desglosarla:

- **SELECT:** Indica qué columnas queremos obtener.
- ``*`` : Significa "todas las columnas" (es un comodín).
- **FROM experimentos:** Especifica la tabla de donde extraer los datos.

Una tabla SQL es como una hoja de cálculo: filas (registros) y columnas (campos). Por ejemplo, una tabla ``experimentos`` podría verse así:

id	nombre	fecha	investigador	resultado
1	Proyecto Fénix	2026-06-01	Blasco	éxito
2	Proyecto Ícaro	2026-06-15	Blasco	fallo
3	Delta-7	2026-07-01	Blasco	éxito

Sintaxis básica

```
-- Obtener todas las columnas de una tabla
SELECT * FROM experimentos;

-- Obtener columnas específicas
SELECT nombre, fecha FROM experimentos;

-- Obtener las primeras 5 filas
SELECT * FROM experimentos LIMIT 5;

-- Ordenar resultados
SELECT * FROM experimentos ORDER BY fecha DESC;
```

¿Por qué es importante?

El `SELECT` es la puerta de entrada a los datos. Sin él, no puedes ver nada. Como detective, tu primera tarea siempre será observar los datos disponibles.

Cuando Elisa vea `SELECT \* FROM experimentos;` incompleto, sabrá que alguien interrumpió al profesor mientras buscaba algo específico.

Enigma SQL #1: Los Registros Perdidos

La policía ha incautado la base de datos del ICB y te ha dado acceso. Usa SQL para responder:

1. ¿Cuántos experimentos hay registrados?
2. Muestra todos los experimentos ordenados por fecha del más reciente al más antiguo.
3. ¿Qué columnas tiene la tabla `experimentos`?

```
-- Escribe tus consultas aquí
```

(Solución en el Apéndice, página 1)

Capítulo 2: La Condición del Sospechoso

—No entiendo —dijo el inspector Rovira, frotándose la barbilla—. El forense insiste en que fue un infarto. Sin signos de violencia.

Elisa estaba frente al ordenador del profesor, revisando los archivos del sistema.

—Mira esto —señaló la pantalla—. Héctor tenía un proyecto secreto. Lo llamó "Proyecto Cerbero".

—¿Qué es?

—No lo sé. Pero hay experimentos que no aparecen en la base de datos principal. Héctor los escondió.

Elisa abrió una terminal y escribió:

```
SELECT * FROM experimentos WHERE proyecto = 'Cerbero';
```

La consulta devolvió cero resultados.

—No hay nada —dijo Rovira.

—O no estamos buscando bien. Déjame pensar.

Elisa sabía que su mentor era meticuloso. Si había creado algo llamado Cerbero, los datos tenían que estar en alguna parte. Quizás no era una tabla independiente, sino registros filtrados por alguna condición.

Recordó una conversación con Héctor semanas atrás: *"Elisa, la condición WHERE es la diferencia entre encontrar la verdad y perderte en el ruido."*

Concepto SQL: WHERE y condiciones

La cláusula **WHERE** filtra filas según una condición. Solo las filas que cumplen la condición aparecen en el resultado.

```
-- Filtrar por un valor exacto
SELECT * FROM experimentos WHERE proyecto = 'Cerbero';

-- Filtrar por número
SELECT * FROM experimentos WHERE id > 10;

-- Múltiples condiciones con AND
SELECT * FROM experimentos
WHERE proyecto = 'Cerbero' AND resultado = 'éxito';

-- Múltiples condiciones con OR
```

```
SELECT * FROM experimentos
WHERE proyecto = 'Cerbero' OR proyecto = 'Fénix';

-- Negar una condición
SELECT * FROM experimentos
WHERE NOT resultado = 'fallo';
```

Operadores de comparación

Operador	Significado	Ejemplo
`=`	Igual a	`WHERE nombre = 'Blasco'`
`<>` o `!=`	Distinto de	`WHERE resultado != 'fallo'`
`>`	Mayor que	`WHERE id > 100`
`<`	Menor que	`WHERE fecha < '2026-07-01'`
`>=`	Mayor o igual	`WHERE version >= 2.0`
`<=`	Menor o igual	`WHERE intentos <= 3`

Operadores especiales

```
-- BETWEEN: valores en un rango
SELECT * FROM experimentos
WHERE fecha BETWEEN '2026-01-01' AND '2026-06-30';

-- IN: valores dentro de una lista
SELECT * FROM experimentos
WHERE investigador IN ('Blasco', 'Riera', 'Valls');

-- LIKE: búsqueda con patrones
SELECT * FROM experimentos
WHERE nombre LIKE 'Proyecto%'; -- Empieza con "Proyecto"

-- IS NULL: valores nulos
SELECT * FROM experimentos
WHERE fecha_fin IS NULL;
```

El Descubrimiento

Elisa probó otra consulta:

```
SELECT * FROM logs_acceso
WHERE usuario = 'hblasco' AND accion LIKE 'CREATE TABLE%';
```

La base de datos devolvió un registro: el profesor había creado una tabla llamada `cerbero\_config` hacía tres meses, a las 2:47 de la madrugada.

—Trabajaba de noche —murmuró Elisa—. ¿Qué escondía, Héctor?

Enigma SQL #2: El Rastro del Profesor

Usando la base de datos del ICB:

1. Encuentra todos los accesos al laboratorio realizados por el Dr. Blasco después de las 22:00

(10 PM).

2. ¿Cuántos experimentos tienen resultado NULL (inconcluso)?
3. Lista los proyectos que contienen la palabra "Cerbera" en su nombre.

```
-- Tus consultas aquí
```

Capítulo 3: Las Tablas del Engaño

Elisa pasó la mañana revisando los logs. La tabla `cerbero\_config` existía, pero cuando intentó consultarla:

```
SELECT * FROM cerbero_config;
```

El resultado fue un error: *"table not found"*.

—La borraron —dijo Rovira.

—No —respondió Elisa—. Está en otra base de datos. Héctor usaba múltiples instancias. Necesito encontrar la conexión.

En el sistema de archivos del profesor encontró un archivo sospechoso: `conexion\_secreta.db`. Lo copió y lo montó en su estación de trabajo.

```
SELECT * FROM cerbero_config;
```

Finalmente, datos:

id	clave	valor	fecha_modificacion	nivel
1	db_host	10.0.0.15	2026-06-01	ALTO
2	db_user	admin_cerbero	2026-06-01	ALTO
3	db_pass	ENCRYPTED	2026-06-01	ALTO
4	umbral_seguridad	0.95	2026-06-15	CRÍTICO
5	umbral_seguridad_anterior	0.50	2026-06-15	ALTO

—El umbral de seguridad se duplicó —observó Elisa—. ¿Por qué?

—¿Qué significa? —preguntó Rovira.

—Cerberero es un sistema de control de acceso biométrico. El umbral determina cuán estricta es la verificación. 0.50 es el estándar. 0.95 es... paranoico.

Rovira frunció el ceño.

—O alguien quería asegurarse de que solo cierta persona pudiera acceder.

Elisa sonrió. Ahora necesitaba cruzar esta información con los registros de acceso.

Concepto SQL: JOINS

Hasta ahora hemos trabajado con una sola tabla. Pero los datos reales están dispersos en múltiples tablas relacionadas. Para combinarlos, usamos **JOIN**.

INNER JOIN

Devuelve solo las filas que tienen correspondencia en ambas tablas:

```
SELECT e.nombre, e.fecha, a.usuario, a.accion
FROM experimentos e
INNER JOIN logs_acceso a ON e.investigador = a.usuario;
```

LEFT JOIN

Devuelve todas las filas de la tabla izquierda, y las correspondientes de la derecha (NULL si no hay match):

```
SELECT e.nombre, c.clave, c.valor
FROM experimentos e
LEFT JOIN cerbero_config c ON e.proyecto = 'Cerbero';
```

Tablas y alias

Usamos alias (`e`, `a`, `c`) para acortar los nombres de las tablas. El patrón es:

```
SELECT alias.columna
FROM tabla1 alias1
JOIN tabla2 alias2 ON alias1.id = alias2.id;
```

Tipos de JOIN

```
-- INNER JOIN: solo coincidencias
SELECT * FROM A INNER JOIN B ON A.id = B.id;

-- LEFT JOIN: todo de A, lo que coincide de B
SELECT * FROM A LEFT JOIN B ON A.id = B.id;

-- RIGHT JOIN: todo de B, lo que coincide de A
SELECT * FROM A RIGHT JOIN B ON A.id = B.id;

-- FULL JOIN: todo de ambas tablas
SELECT * FROM A FULL JOIN B ON A.id = B.id;
```

La Conexión

Elisa unió las tablas de acceso con la configuración de Cerbero:

```
SELECT a.usuario, a.fecha, a.puerta, c.valor as umbral
FROM logs_acceso a
JOIN cerbero_config c ON c.clave = 'umbral_seguridad'
WHERE a.fecha > c.fecha_modificacion
      AND a.acceso_concedido = 1
ORDER BY a.fecha DESC;
```

El resultado mostraba que solo dos personas accedieron al laboratorio después del cambio de umbral: el Dr. Blasco y una cuenta llamada `mroble`.

—Marta Robles —leyó Rovira—. La asistente del profesor.

—Llámosla para interrogarla —dijo Elisa.

Pero cuando marcaron su número, la llamada entró directamente al buzón de voz.

Enigma SQL #3: ¿Quién más sabía?

Usando joins entre las tablas `personal`, `accesos` y `experimentos`:

1. ¿Qué otros investigadores tenían acceso al proyecto Cerbero?
2. Lista todos los accesos al laboratorio 7 ocurridos durante el mes de junio de 2026, mostrando el nombre del empleado y la puerta.
3. Encuentra a las personas que accedieron al laboratorio pero NO están registradas como investigadoras en el proyecto Cerbero.

```
-- Tus consultas aquí
```

Capítulo 4: El Grupo que Miente

Marta Robles apareció al día siguiente. Tenía 28 años, ojos inquietos y una respuesta preparada para todo.

—Sí, trabajaba con el doctor Blasco. Sí, sabía de Cerbero. No, no sé quién lo mató.

Elisa la observaba desde el otro lado de la mesa. Había algo en su lenguaje corporal que no encajaba. Demasiado controlada.

—¿Por qué accediste al laboratorio a las 3 de la madrugada del 15 de julio? —preguntó Elisa.

—Me olvidé el teléfono.

—Tu teléfono estaba en tu bolso, Marta. Lo vi en las imágenes de seguridad.

Marta tragó saliva.

—Está bien. Fui a revisar unos datos. El doctor me pidió que verificara los logs de Cerbero.

—¿A las 3 de la mañana?

—Era... urgente.

Después de la entrevista, Elisa volvió a su terminal. Algo no cuadraba. Decidió analizar los patrones de acceso de Marta usando agregaciones SQL.

Concepto SQL: GROUP BY y funciones de agregación

Hasta ahora hemos visto filas individuales. Pero a menudo necesitamos **resumir** datos: contar, sumar, promediar. Para eso usamos **GROUP BY** junto con funciones de agregación.

Funciones de agregación

```
-- COUNT: contar filas
SELECT COUNT(*) FROM accesos;
SELECT COUNT(DISTINCT usuario) FROM accesos;

-- SUM: sumar valores numéricos
SELECT SUM(duracion_minutos) FROM sesiones;

-- AVG: promedio
SELECT AVG(duracion_minutos) FROM sesiones;

-- MAX / MIN: valores máximo y mínimo
SELECT MAX(intentos_fallidos) FROM accesos;
SELECT MIN(intentos_fallidos) FROM accesos;
```

GROUP BY: Agrupar datos

```
-- Contar accesos por usuario
SELECT usuario, COUNT(*) as total_accesos
FROM accesos
GROUP BY usuario;

-- Promedio de duración por día
SELECT DATE(fecha) as dia, AVG(duracion) as promedio
FROM sesiones
GROUP BY DATE(fecha);

-- Múltiples niveles de agrupación
SELECT usuario, puerta, COUNT(*) as veces
FROM accesos
GROUP BY usuario, puerta;
```

HAVING: Filtrar grupos

WHERE filtra filas **antes** de agrupar. HAVING filtra grupos **después** de agrupar:

```
-- Usuarios con más de 50 accesos
SELECT usuario, COUNT(*) as total
FROM accesos
GROUP BY usuario
HAVING COUNT(*) > 50;

-- Días con más de 10 accesos fallidos
SELECT DATE(fecha) as dia, COUNT(*) as fallos
FROM accesos
WHERE acceso_concedido = 0
GROUP BY DATE(fecha)
HAVING COUNT(*) > 10;
```

El Patrón

Elisa agrupó los accesos de Marta y encontró algo extraño:

```
SELECT usuario,
       DATE(fecha) as dia,
       COUNT(*) as accesos,
       MIN(fecha) as primero,
       MAX(fecha) as ultimo
FROM accesos
WHERE usuario = 'mrobles'
GROUP BY usuario, DATE(fecha)
HAVING COUNT(*) > 3
ORDER BY dia;
```

El resultado mostraba que Marta accedía al laboratorio en bloques: varios accesos en pocos minutos, como si estuviera entrando y saliendo repetidamente. O como si estuviera... borrando algo.

Pero además, había un detalle más inquietante: los días que Marta tenía más accesos coincidían exactamente con los días en que el Dr. Blasco estaba fuera del instituto en conferencias.

—Marta sabía cuándo Héctor no estaría —murmuró Elisa—. Y usaba ese tiempo para... ¿qué?

Enigma SQL #4: La Agenda Oculta

1. ¿Cuántos accesos al laboratorio 7 realizó cada empleado durante el mes de junio? Ordena de mayor a menor.
2. ¿Qué días hubo más de 5 accesos fallidos en total (todos los usuarios)?
3. Calcula el promedio de accesos por hora del día. ¿Hay una hora con actividad inusual?

```
-- Tus consultas aquí
```

Capítulo 5: La Fecha del Crimen

El forense entregó el informe definitivo: muerte por paro cardíaco inducido por una toxina llamada tetrodotoxina. Alguien la había administrado horas antes del deceso.

—Es una toxina que se encuentra en el pez globo —explicó Rovira—. Es indetectable en autopsias rutinarias. Si no la hubieran buscado específicamente...

—Alguien sabía medicina —completó Elisa.

Pero había un problema: según los registros de acceso, el Dr. Blasco estuvo solo en su laboratorio desde las 22:00 hasta que lo encontraron muerto a las 5:30. Nadie entró. Nadie salió.

—¿Cómo administraron la toxina? —preguntó Elisa en voz alta.

Revisó los logs de la base de datos con más atención. Algo llamó su atención: el profesor había recibido un paquete del almacén de químicos a las 21:45.

```
SELECT * FROM envios
WHERE destinatario = 'Héctor Blasco'
AND fecha BETWEEN '2026-07-14 20:00' AND '2026-07-14 22:00';
```

id	remitente	destinatario	contenido	fecha_envio
8932	Almacén QCA	H. Blasco	Muestra T-47	2026-07-14 21:30

—¿Qué es la muestra T-47? —preguntó Elisa.

Hizo otra consulta:

```
SELECT * FROM inventario_quimico WHERE codigo LIKE 'T-%';
```

No había registros. Alguien había borrado la entrada.

—Marta —dijo Rovira.

—O alguien con acceso a la base de datos —corrigió Elisa.

Entonces recordó que la base de datos registraba fechas de modificación. Si alguien manipuló los datos después del crimen...

Concepto SQL: Funciones de fecha y hora

Las fechas son cruciales en investigación forense. SQL ofrece potentes funciones para trabajar con ellas.

Tipos de datos temporales

```
-- DATE: solo fecha (2026-07-14)
-- TIME: solo hora (21:45:00)
-- DATETIME / TIMESTAMP: fecha y hora (2026-07-14 21:45:00)
```

Funciones de fecha

```
-- Fecha actual
SELECT DATE('now');
SELECT DATETIME('now');

-- Extraer partes
SELECT strftime('%Y', fecha) as año FROM tabla; -- 2026
SELECT strftime('%m', fecha) as mes FROM tabla; -- 07
SELECT strftime('%d', fecha) as día FROM tabla; -- 14
SELECT strftime('%H', fecha) as hora FROM tabla; -- 21
SELECT strftime('%w', fecha) as día_sem FROM tabla; -- 0=domingo

-- Operaciones con fechas
SELECT DATE(fecha, '+1 day') as mañana FROM tabla;
SELECT DATE(fecha, '-7 days') as semana_pasada FROM tabla;
SELECT DATETIME(fecha, '+2 hours') FROM tabla;
```

WHERE con fechas

```
-- Rango de fechas
SELECT * FROM accesos
WHERE fecha >= '2026-07-14 22:00'
      AND fecha < '2026-07-15 06:00';

-- Filtrar por día de semana
SELECT * FROM accesos
WHERE strftime('%w', fecha) = '0'; -- Domingos

-- Filtrar por hora
SELECT * FROM accesos
WHERE strftime('%H', fecha) BETWEEN '22' AND '23'; -- 10-11 PM
```

La Anomalía

Elisa ejecutó una consulta clave:

```
SELECT tabla, fecha_modificacion, usuario
FROM logs_modificaciones
WHERE tabla = 'inventario_quimico'
      AND fecha_modificacion > '2026-07-15 05:30'
ORDER BY fecha_modificacion DESC;
```

La consulta reveló que alguien modificó la tabla `inventario\_quimico` a las **6:15 AM del 15 de julio** — 45 minutos después del hallazgo del cuerpo, cuando el laboratorio estaba sellado. El usuario era `hblasco`.

- Pero Héctor estaba muerto —susurró Elisa.
 - Alguien usó su cuenta —dijo Rovira.
 - O alguien sabía que iba a morir y programó la modificación.
-

Enigma SQL #5: La Línea de Tiempo

1. Encuentra todos los registros de la tabla `logs\_modificaciones` del día 15 de julio de 2026, ordenados por hora.
2. ¿Hubo modificaciones a la base de datos entre las 22:00 del 14 de julio y las 6:00 del 15 de julio, antes del hallazgo del cuerpo?
3. ¿Cuánto tiempo pasó entre la última modificación hecha por el usuario `hblasco` y la primera modificación hecha por `mroble`?

```
-- Tus consultas aquí
```

Capítulo 6: La Subconsulta Asesina

—La cuenta de Héctor fue usada después de su muerte —dijo Elisa frente al panel de investigadores—. Pero no fue Marta. La IP de conexión es diferente.

—¿Qué IP? —preguntó Rovira.

—Una dirección externa. Alguien se conectó desde fuera del instituto.

—¿Puedes rastrearla?

Elisa asintió. Primero necesitaba aislar todas las conexiones sospechosas de las legítimas. Decidió usar subconsultas.

Concepto SQL: Subconsultas

Una **subconsulta** es una consulta dentro de otra consulta. Sirve para realizar operaciones en múltiples pasos dentro de una sola instrucción SQL.

Subconsulta en WHERE

```
-- Encontrar usuarios que accedieron después del crimen
SELECT * FROM accesos
WHERE usuario IN (
    SELECT DISTINCT usuario
    FROM logs_modificaciones
    WHERE fecha_modificacion > '2026-07-15 05:30'
);
```

Subconsulta con comparación

```
-- Accesos con duración mayor al promedio
SELECT * FROM accesos
WHERE duracion > (
    SELECT AVG(duracion) FROM accesos
);
```

Subconsulta en SELECT

```
-- Mostrar cada acceso junto al total de accesos del usuario
SELECT a.*, (
    SELECT COUNT(*)
    FROM accesos a2
    WHERE a2.usuario = a.usuario
) as total_usuario
```

```
FROM accesos a;
```

EXISTS / NOT EXISTS

```
-- Usuarios que tienen al menos un acceso nocturno
SELECT DISTINCT usuario FROM personal p
WHERE EXISTS (
    SELECT 1 FROM accesos a
    WHERE a.usuario = p.codigo
        AND strftime('%H', a.fecha) BETWEEN '22' AND '06'
);

-- Usuarios sin accesos después del crimen
SELECT * FROM personal p
WHERE NOT EXISTS (
    SELECT 1 FROM accesos a
    WHERE a.usuario = p.codigo
        AND a.fecha > '2026-07-15 05:30'
);
```

Subconsultas anidadas

```
-- La IP con más accesos sospechosos
SELECT ip, COUNT(*) as total
FROM conexiones
WHERE ip IN (
    SELECT DISTINCT ip_origen
    FROM logs_modificaciones
    WHERE fecha > '2026-07-15 05:30'
        AND usuario NOT IN (
            SELECT codigo FROM personal WHERE autorizado = 1
        )
)
GROUP BY ip
ORDER BY total DESC
LIMIT 1;
```

La IP Fantasma

Elisa ejecutó una subconsulta para encontrar la IP del intruso:

```
SELECT ip_origen, COUNT(*) as operaciones
FROM logs_modificaciones
WHERE fecha_modificacion > '2026-07-15 05:30'
    AND usuario = 'hblasco'
GROUP BY ip_origen;
```

Solo una IP: `185.23.47.101`.

—Está en Rusia —dijo Rovira—. O al menos usa un VPN que lo parece.

—No —dijo Elisa—. Mira más de cerca. La IP aparece antes del crimen también, pero con otro usuario.

Ejecutó:

```
SELECT * FROM conexiones
WHERE ip_origen = '185.23.47.101'
      AND fecha < '2026-07-15 05:30'
ORDER BY fecha DESC
LIMIT 5;
```

Las conexiones mostraban al usuario `mrobles`. Marta Robles se conectaba desde la misma IP sospechosa.

—Marta no está diciendo toda la verdad —concluyó Rovira.

—O alguien más usa la IP de Marta —dijo Elisa.

Enigma SQL #6: La Red Oculta

1. Encuentra todos los usuarios que se conectaron desde IPs externas (que no son del rango 10.0.0.x) después del crimen.
2. ¿Cuántas IPs diferentes usó Marta Robles en los últimos 30 días?
3. Lista los usuarios cuyos accesos nocturnos (22:00-06:00) superan el promedio de accesos nocturnos del instituto.

```
-- Tus consultas aquí
```

Capítulo 7: El Índice Oculto

—Necesito mirar el disco duro del servidor —dijo Elisa.

—¿Qué esperas encontrar? —preguntó Rovira.

—Archivos residuales. Datos que alguien intentó borrar pero dejó rastros.

Elisa accedió al servidor de bases de datos del ICB. El sistema operativo mostraba un archivo sospechoso en `/tmp/datos\_recuperados.sql`. Lo abrió.

Era un script SQL que creaba una tabla llamada `transacciones\_ocultas`.

—Alguien volcó datos aquí y borró la tabla original —murmuró Elisa—. Pero el archivo temporal sobrevivió.

Ejecutó el script y consultó la tabla:

```
SELECT * FROM transacciones_ocultas;
```

La tabla tenía 10,847 registros. Transferencias bancarias. Del laboratorio Cerbero a cuentas en paraísos fiscales. La más reciente: 200,000 euros transferidos el 14 de julio a las 14:30.

—El día antes del asesinato —dijo Rovira.

—Y el beneficiario es...

Elisa ejecutó:

```
SELECT DISTINCT beneficiario, SUM(monto) as total
FROM transacciones_ocultas
GROUP BY beneficiario
ORDER BY total DESC;
```

El primer nombre en la lista la dejó sin aliento.

Concepto SQL: Índices y optimización

Hasta ahora nos hemos enfocado en la sintaxis. Pero cuando trabajas con miles o millones de registros, la **velocidad** importa. Ahí entran los **índices**.

¿Qué es un índice?

Un índice es como el índice alfabético de un libro: en lugar de hojear todas las páginas para encontrar una palabra, vas directamente a la página.

```
-- Crear un índice en una columna
CREATE INDEX idx_accesos_usuario ON accesos(usuario);

-- Índice compuesto (múltiples columnas)
CREATE INDEX idx_accesos_fecha_usuario
```

```

ON accesos(fecha, usuario);

-- Índice único (valores no repetidos)
CREATE UNIQUE INDEX idx_personal_email
ON personal(email);

-- Ver los índices de una tabla
SELECT * FROM sqlite_master
WHERE type = 'index' AND tbl_name = 'accesos';

```

¿Cuándo usa SQL un índice?

Las consultas con WHERE, JOIN, ORDER BY y GROUP BY se benefician de los índices:

```

-- Sin índice: revisa todas las filas (table scan)
SELECT * FROM transacciones WHERE beneficiario = 'X';

-- Con índice: busca directamente (index seek)
-- (crea el índice primero)
CREATE INDEX idx_beneficiario ON transacciones(beneficiario);

```

EXPLAIN QUERY PLAN

Para ver si una consulta usa índices:

```

EXPLAIN QUERY PLAN
SELECT * FROM transacciones
WHERE beneficiario = 'X' AND monto > 1000;

```

Cuidado con los índices

```

-- Los índices aceleran lecturas pero ralentizan escrituras
-- No indexes todo, solo lo que consultas frecuentemente

-- Las funciones en columnas pueden hacer que los índices
-- no se usen (a menos que sean índices funcionales)
SELECT * FROM accesos WHERE strftime('%Y', fecha) = '2026';
-- Mejor:
SELECT * FROM accesos
WHERE fecha >= '2026-01-01' AND fecha < '2027-01-01';

```

La Tabla Oculta

Elisa se quedó mirando el resultado de la consulta. El nombre en la cima de la lista de beneficiarios era: **ICB - Instituto de Ciberseguridad de Barcelona**.

—El dinero iba al propio instituto —dijo—. Pero a una cuenta que no aparece en los registros oficiales.

—Una cuenta fantasma —completó Rovira.

—Héctor descubrió el desvío de fondos. Por eso lo mataron.

Pero entonces, ¿quién estaba detrás? Alguien con suficiente acceso para crear transacciones ocultas y borrar evidencias. Alguien desde dentro.

Enigma SQL #7: El Rastro del Dinero

1. ¿Cuántas transacciones sospechosas hay por cada año?
2. Crea un índice en la columna `beneficiario` de la tabla `transacciones\_ocultas` y verifica si la consulta `SELECT \* FROM transacciones\_ocultas WHERE beneficiario = 'ICB'` lo usa (con EXPLAIN QUERY PLAN).
3. Encuentra la transacción de mayor monto y todos sus detalles.

```
-- Tus consultas aquí
```

Capítulo 8: La Vista del Pasado

—El dinero iba al ICB —confirmó el subdirector Vilaró, un hombre calvo de 55 años con traje impecable—. Pero no es lo que parece. Era un fondo de contingencia para operaciones de ciberseguridad. Aprobado por el consejo.

—¿Por qué estaba oculto? —preguntó Elisa.

—No estaba oculto. Estaba... protegido. Si alguien accedía a la información sin autorización, los datos se auto-borrarían.

Elisa no le creyó. Volvió a su terminal y decidió crear una estructura de consulta que simplificara su investigación: una **vista** (VIEW).

Una vista es como una consulta guardada que puedes usar como si fuera una tabla. No almacena datos, sino la definición de cómo obtenerlos.

Concepto SQL: Vistas (VIEWS)

Una **vista** es una tabla virtual basada en el resultado de una consulta. Simplifica consultas complejas y añade una capa de seguridad.

Crear una vista

```
-- Vista simple
CREATE VIEW v_accesos_recientes AS
SELECT usuario, fecha, puerta, acceso_concedido
FROM accesos
WHERE fecha > DATE('now', '-30 days')
ORDER BY fecha DESC;

-- Usar la vista como una tabla
SELECT * FROM v_accesos_recientes;
SELECT usuario, COUNT(*) FROM v_accesos_recientes
WHERE acceso_concedido = 0
GROUP BY usuario;
```

Vista con JOINS

```
CREATE VIEW v_investigacion_completa AS
SELECT a.id, a.fecha, a.usuario, p.nombre as nombre_personal,
       p.departamento, a.puerta, a.acceso_concedido
FROM accesos a
JOIN personal p ON a.usuario = p.codigo;
```

Vista con agregaciones

```
CREATE VIEW v_estadisticas_accesos AS
SELECT usuario,
       COUNT(*) as total_accesos,
       SUM(CASE WHEN acceso_concedido = 0 THEN 1 ELSE 0 END) as fallos,
       ROUND(AVG(CAST(strftime('%H', fecha) AS REAL)), 1) as hora_promedio
FROM accesos
GROUP BY usuario;
```

¿Vista o tabla?

Vista	Tabla
No ocupa espacio (solo la definición)	Ocupa espacio
Siempre datos actualizados	Datos estáticos hasta que se modifican
No se puede indexar directamente	Se puede indexar
Útil para seguridad (ocultar columnas)	Expone toda la estructura

Eliminar una vista

```
DROP VIEW IF EXISTS v_investigacion_completa;
```

La Vista que lo Cambió Todo

Elisa creó una vista que reunía toda la información dispersa:

```
CREATE VIEW v_caso_blasco AS
SELECT a.fecha as fecha_acceso, a.usuario, a.puerta,
       p.nombre, p.departamento, p.cargo,
       t.monto, t.beneficiario, t.fecha as fecha_transaccion
FROM accesos a
JOIN personal p ON a.usuario = p.codigo
LEFT JOIN transacciones_ocultas t ON t.fecha BETWEEN
    datetime(a.fecha, '-1 hour') AND datetime(a.fecha, '+1 hour')
WHERE a.laboratorio = 7
    AND a.fecha > '2026-06-01';
```

La vista reveló algo que las consultas individuales no mostraban: Marta Robles aparecía en el laboratorio exactamente 15 minutos antes de cada transacción grande. Y el subdirector Vilaró... nunca aparecía en los accesos, pero sus credenciales de administrador se usaban para aprobar las transferencias.

—Vilaró es el jefe de Marta —dijo Rovira—. Podría estar encubriéndola.

—O ella podría estar trabajando para él —respondió Elisa.

Enigma SQL #8: La Investigación Simplificada

1. Crea una vista que muestre todos los accesos al laboratorio 7 de empleados del departamento de "Sistemas" durante junio y julio de 2026.
2. Usando la vista `v\_investigacion\_completa` (o creando la tuya), encuentra qué empleados

accedieron tanto al laboratorio 7 como al almacén de químicos en el mismo día.

3. ¿Cuántos accesos realizó cada departamento al laboratorio 7? Usa una vista para responder.

```
-- Tus consultas aquí
```

Capítulo 9: La Transacción Perfecta

Elisa tenía suficiente para un arresto, pero necesitaba la pieza final: la conexión entre Vilaró y el laboratorio. No había registros de acceso físico, pero tenía que haber alguna comunicación.

Revisó los logs de la base de datos del sistema financiero. Encontró algo: transacciones realizadas desde la cuenta de Vilaró que coincidían exactamente con accesos a la base de datos desde la IP del laboratorio.

—Hacía transferencias desde el laboratorio —dijo Elisa—. Las programaba y luego borraba los logs.

Pero faltaba algo. Las transacciones se realizaban pero no se completaban. Elisa abrió los registros de transacciones de la base de datos.

Concepto SQL: Transacciones

Una **transacción** es un conjunto de operaciones que se ejecutan como una unidad. O se completan todas, o ninguna. Es la base de la integridad de los datos.

ACID

Las bases de datos relacionales garantizan cuatro propiedades:

- **Atomicidad:** Todo se ejecuta o nada se ejecuta.
- **Consistencia:** Los datos siempre están en un estado válido.
- **Aislamiento:** Las transacciones concurrentes no interfieren.
- **Durabilidad:** Una vez confirmada, la transacción persiste.

BEGIN, COMMIT, ROLLBACK

```
-- Iniciar una transacción
BEGIN TRANSACTION;

-- Operaciones
UPDATE cuentas SET saldo = saldo - 1000 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 1000 WHERE id = 2;

-- Si todo está bien, confirmar
COMMIT;

-- Si algo sale mal, deshacer
ROLLBACK;
```

Ejemplo de aplicación bancaria

```
BEGIN TRANSACTION;

-- Verificar saldo suficiente
SELECT saldo FROM cuentas WHERE id = 1;
-- Resultado: 5000

-- Descontar
UPDATE cuentas SET saldo = saldo - 2000 WHERE id = 1;

-- Acreditar
UPDATE cuentas SET saldo = saldo + 2000 WHERE id = 2;

-- Verificar consistencia
SELECT SUM(saldo) FROM cuentas;
-- Debe ser el mismo valor que antes

COMMIT;
```

Transacciones e integridad

```
-- Si ocurre un error, todo se deshace
BEGIN TRANSACTION;
INSERT INTO logs VALUES ('transferencia_iniciada', datetime('now'));
UPDATE cuentas SET saldo = saldo - 5000 WHERE id = 1;
-- Error: la cuenta 2 no existe
UPDATE cuentas SET saldo = saldo + 5000 WHERE id = 999;
ROLLBACK;
-- La cuenta 1 no perdió el dinero
```

SAVEPOINT

```
-- Puntos de guardado dentro de una transacción
BEGIN TRANSACTION;
INSERT INTO logs VALUES ('inicio', datetime('now'));
SAVEPOINT despues_log;
UPDATE cuentas SET saldo = 0 WHERE id = 1;
-- Ups, no quería eso
ROLLBACK TO SAVEPOINT despues_log;
-- El log se mantiene, pero la actualización se deshizo
COMMIT;
```

La Transacción Manipulada

Elisa encontró algo extraño en los logs de transacciones:

```
SELECT * FROM logs_transacciones
WHERE usuario = 'vilario'
      AND fecha BETWEEN '2026-07-14 20:00' AND '2026-07-14 23:00'
ORDER BY fecha;
```

Una transacción llamada `PAGO\_PROVEEDOR\_AGO` se inició a las 22:15 y se confirmó a las 22:17. Pero no había registros intermedios.

—Dos minutos —murmuró Elisa—. Es muy rápido para una transferencia internacional.

Revisó los datos de la transferencia:

Campo	Valor
Origen	ICB - Cuenta General
Destino	SwissCredit Bank - Cuenta 4423-XX
Monto	500,000 EUR
Concepto	Servicios de consultoría
Aprobador	M. Robles
Ejecutor	H. Blasco (?)

—Usaron la cuenta de Héctor para ejecutar la transacción —dijo Elisa—. Y la aprobó Marta.

—Pero si Héctor estaba muerto a las 22:15... —dijo Rovira.

—Exacto. La transacción se ejecutó después de su muerte. O alguien la programó, o... Héctor seguía vivo a las 22:17.

Elisa sintió un escalofrío.

Enigma SQL #9: La Cadena de Transacciones

1. Simula una transacción que transfiera 500,000 EUR de una cuenta a otra. Escribe el SQL completo con BEGIN, COMMIT y verificaciones.

2. ¿Cuántas transacciones en el sistema tienen un tiempo entre inicio y confirmación menor a 5 minutos? Esto podría indicar automatización.

3. Encuentra todas las transacciones que se ejecutaron con la cuenta de un usuario fallecido (después de su hora de muerte registrada).

```
-- Tus consultas aquí
```

Capítulo 10: La Función del Silencio

La evidencia se acumulaba, pero Elisa sentía que algo no encajaba. Marta Robles tenía acceso a las cuentas, cierto. Vilaró tenía el motivo, también. Pero ¿quién había programado la transacción? ¿Y cómo sabían que Héctor estaría muerto a las 22:15?

Decidió analizar los datos de manera más profunda. Había un campo que había ignorado: las notas de las transacciones. Algunas contenían códigos extraños.

```
SELECT id, monto, concepto, notas
FROM transacciones_ocultas
WHERE notas IS NOT NULL
AND notas != '';
```

Entre las notas encontró mensajes codificados. Pero no eran al azar; seguían un patrón. Cada nota era una concatenación de iniciales y números. Como si fueran... instrucciones.

—No son notas —dijo Elisa de repente—. Son comandos. Instrucciones para alguien más.

Concepto SQL: Funciones de cadena y manipulación de datos

SQL no solo sirve para consultar datos, sino para transformarlos. Las funciones de cadena son esenciales para limpiar, extraer y manipular información.

Funciones de cadena

```
-- Longitud de un texto
SELECT LENGTH('Hola Mundo'); -- 10

-- Convertir a mayúsculas/minúsculas
SELECT UPPER('hola'); -- HOLA
SELECT LOWER('HOLA'); -- hola

-- Extraer parte de una cadena (SQLite)
SELECT SUBSTR('Hola Mundo', 1, 4); -- Hola
SELECT SUBSTR('Hola Mundo', 6, 5); -- Mundo

-- Buscar posición
SELECT INSTR('Hola Mundo', 'Mundo'); -- 6

-- Concatenar
SELECT 'Hola' || ' ' || 'Mundo'; -- Hola Mundo

-- Reemplazar
```

```

SELECT REPLACE('Hola Mundo', 'Mundo', 'SQL'); -- Hola SQL

-- Trim (eliminar espacios)
SELECT TRIM('  Hola '); -- Hola
SELECT LTRIM('  Hola'); -- Hola
SELECT RTRIM('Hola '); -- Hola

```

Extracción de información

```

-- Obtener iniciales
SELECT SUBSTR(nombre, 1, 1) || SUBSTR(apellido, 1, 1) as iniciales
FROM personal;

-- Extraer dominio de email
SELECT email, SUBSTR(email, INSTR(email, '@') + 1) as dominio
FROM personal;

-- Formatear fechas
SELECT strftime('%d/%m/%Y %H:%M', fecha) as fecha_formateada
FROM accesos;

```

CASE: Condicionales en SQL

```

-- Clasificar accesos
SELECT usuario, fecha,
CASE
    WHEN strftime('%H', fecha) BETWEEN '06' AND '12' THEN 'Mañana'
    WHEN strftime('%H', fecha) BETWEEN '12' AND '18' THEN 'Tarde'
    WHEN strftime('%H', fecha) BETWEEN '18' AND '22' THEN 'Noche'
    ELSE 'Madrugada'
END as turno
FROM accesos;

-- Categorizar montos
SELECT monto,
CASE
    WHEN monto < 1000 THEN 'Bajo'
    WHEN monto BETWEEN 1000 AND 10000 THEN 'Medio'
    WHEN monto BETWEEN 10000 AND 100000 THEN 'Alto'
    ELSE 'Crítico'
END as nivel
FROM transacciones_ocultas;

```

COALESCE e IFNULL

```

-- Valores por defecto para NULL
SELECT nombre, COALESCE(telefono, 'No registrado') as telefono
FROM personal;

SELECT nombre, IFNULL(email, 'sin-email@instituto.com') as email
FROM personal;

```

El Mensaje Oculto

Elisa aplicó funciones de cadena a las notas de las transacciones:

```
SELECT id, monto,  
       SUBSTR(notas, 1, 3) as codigo,  
       SUBSTR(notas, INSTR(notas, ':') + 1) as instruccion  
FROM transacciones_ocultas  
WHERE notas LIKE 'CMD:%';
```

Los resultados mostraban códigos como `CMD:TRANSFERIR\_ARCHIVOS`, `CMD:BORRAR\_LOGS`, `CMD:ACTIVAR\_PROTOCOLO`.

—Son órdenes —dijo Elisa—. Alguien daba instrucciones a través de las transacciones bancarias.

—¿Quién? —preguntó Rovira.

Elisa modificó la consulta:

```
SELECT usuario, COUNT(*) as ordenes_dadas  
FROM transacciones_ocultas  
WHERE notas LIKE 'CMD:%'  
GROUP BY usuario  
ORDER BY ordenes_dadas DESC;
```

El resultado: **M. Robles** era la única que insertaba notas con comandos.

—Marta no solo sabía —concluyó Elisa—. Marta ejecutaba las órdenes. Pero ¿de quién?

Enigma SQL #10: Decodificando el Mensaje

1. Extrae los primeros 4 caracteres de cada concepto de transacción. ¿Hay un patrón visible?
2. Clasifica las transacciones en categorías según el monto (bajo < 1000, medio 1000-50000, alto > 50000). ¿Cuántas hay de cada tipo?
3. Crea una consulta que muestre el nombre completo del usuario (Nombre + Apellido) y el total de transacciones que realizó.

```
-- Tus consultas aquí
```

Capítulo 11: El Trigger del Miedo

—Marta no es la mente maestra —dijo Elisa frente al panel—. Es una ejecutora. Alguien le daba órdenes a través de las transacciones. Pero necesito probar que no actuaba sola.

—¿Cómo? —preguntó Rovira.

—Los triggers. Héctor configuró triggers en la base de datos para auditar cambios críticos. Si alguien modificó datos sensibles, el trigger debió registrar algo.

—Pero dijiste que borraron los logs.

—Los logs de aplicación. Pero los triggers de base de datos operan a nivel interno. Son más difíciles de eliminar.

Elisa accedió al esquema de la base de datos y encontró algo:

```
SELECT * FROM sqlite_master WHERE type = 'trigger';
```

Había un trigger llamado `trg\_auditar\_transferencias`.

Concepto SQL: Triggers

Un **trigger** es un procedimiento que se ejecuta automáticamente cuando ocurre un evento en una tabla (INSERT, UPDATE, DELETE). Es como un guardián silencioso.

Sintaxis básica

```
CREATE TRIGGER nombre_trigger
[BEFORE | AFTER] [INSERT | UPDATE | DELETE] ON tabla
[FOR EACH ROW]
BEGIN
    -- instrucciones SQL
END;
```

Ejemplo: Auditoría de cambios

```
-- Tabla de auditoría
CREATE TABLE auditoria_transferencias (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    accion TEXT,
    usuario TEXT,
    monto_anterior REAL,
    monto_nuevo REAL,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Trigger después de UPDATE
CREATE TRIGGER trg_auditar_transferencias
AFTER UPDATE ON transacciones
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_transferencias
        (accion, usuario, monto_anterior, monto_nuevo, fecha)
    VALUES ('UPDATE', OLD.usuario_mod, OLD.monto, NEW.monto,
        datetime('now'));
END;
```

BEFORE vs AFTER

```
-- BEFORE: validar datos antes de insertar
CREATE TRIGGER trg_validar_monto
BEFORE INSERT ON transacciones
FOR EACH ROW
BEGIN
    SELECT CASE
        WHEN NEW.monto <= 0 THEN RAISE(ABORT, 'El monto debe ser positivo')
        WHEN NEW.beneficiario IS NULL THEN RAISE(ABORT, 'Beneficiario requerido')
    END;
END;

-- AFTER: registrar después de la operación
CREATE TRIGGER trg_log_insercion
AFTER INSERT ON transacciones
FOR EACH ROW
BEGIN
    INSERT INTO logs_eventos (evento, detalle, fecha)
    VALUES ('INSERT', 'Transacción #' || NEW.id, datetime('now'));
END;
```

OLD y NEW

Dentro del trigger, puedes referirte a los valores antes y después del cambio:

```
-- OLD: valores antes del cambio (UPDATE, DELETE)
-- NEW: valores después del cambio (INSERT, UPDATE)

CREATE TRIGGER trg_bitacora_cambios
AFTER UPDATE ON personal
FOR EACH ROW
BEGIN
    INSERT INTO bitacora (tabla, campo, valor_anterior, valor_nuevo, fecha)
    VALUES ('personal', 'cargo', OLD.cargo, NEW.cargo, datetime('now'));
END;
```

Prevenir cambios no autorizados

```
CREATE TRIGGER trg_prevenir_borrado_logs
BEFORE DELETE ON logs_modificaciones
```

```

FOR EACH ROW
BEGIN
    SELECT CASE
        WHEN OLD.fecha > datetime('now', '-1 day')
        THEN RAISE(ABORT, 'No se pueden borrar logs recientes')
    END;
END;

```

El Trigger de Héctor

Elisa examinó el trigger que Héctor había creado:

```

CREATE TRIGGER trg_auditar_transferencias
AFTER INSERT ON transacciones
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_oculta
        (transaccion_id, usuario, monto, ip_origen, fecha, hash)
    VALUES (NEW.id,
        (SELECT usuario FROM conexiones WHERE ip = NEW.ip_creacion ORDER BY fecha DESC LIMIT 1),
        NEW.monto, NEW.ip_creacion, datetime('now'),
        hex(randomblob(16)));
END;

```

Héctor había creado una tabla `auditoria\_oculta` que registraba cada transacción junto con el usuario real que la realizó, identificado por su IP. Y el trigger no se podía borrar fácilmente porque estaba protegido.

Elisa consultó la tabla:

```

SELECT * FROM auditoria_oculta
WHERE fecha > '2026-07-01'
ORDER BY fecha;

```

Los registros mostraban que cada transacción de Marta Robles estaba vinculada a una IP: la misma IP que aparecía en las conexiones del subdirector Vilaró. Pero había algo más: en las transacciones realizadas después de las 22:00 del 14 de julio, el usuario registrado era `hblasco`, pero la IP era la de Vilaró.

—Vilaró usó la cuenta de Héctor —dijo Elisa—. Y Marta ejecutó las órdenes. Pero el trigger lo registró todo.

—Tenemos el caso —dijo Rovira.

—Casi. Necesito saber por qué.

Enigma SQL #11: El Guardián Silencioso

1. Crea un trigger que registre en una tabla `log\_eliminaciones` cada vez que alguien intente borrar un registro de la tabla `transacciones`.
2. ¿Qué información registró el trigger `trg\_auditar\_transferencias` sobre las transacciones del 14 de julio?
3. Crea un trigger que impida que alguien modifique el campo `monto` de una transacción si

esta ya fue confirmada (campo `estado = 'CONFIRMADA'`).

```
-- Tus consultas aquí
```

Capítulo 12: La Normalización del Caos

Elisa tenía la evidencia técnica, pero aún no entendía el motivo. ¿Por qué Vilaró, un subdirector bien pagado, arriesgaría su carrera y su libertad?

Decidió analizar la estructura de datos del instituto. Necesitaba entender cómo se relacionaban todas las piezas. La base de datos del ICB era un laberinto de tablas mal diseñadas, redundancias y datos inconsistentes.

—Esto es un desastre —murmuró Elisa—. Los mismos datos en múltiples tablas, nombres inconsistentes...

—¿Importa eso? —preguntó Rovira.

—Importa más de lo que crees. Una base de datos bien diseñada revela la verdad. Una mal diseñada la oculta.

Concepto SQL: Normalización

La **normalización** es el proceso de organizar datos para reducir redundancia y mejorar integridad. Se divide en "formas normales".

Primera Forma Normal (1NF)

Cada celda debe contener un solo valor. No listas ni arreglos.

```
-- MAL: múltiples valores en una celda
CREATE TABLE proyectos (
  id INT,
  investigador TEXT, -- "Elisa, Marta, Héctor"
  PRIMARY KEY (id)
);

-- BIEN: cada valor en su propia fila
CREATE TABLE proyectos (
  id INT,
  investigador TEXT,
  PRIMARY KEY (id, investigador)
);

-- O mejor: tabla separada
CREATE TABLE proyecto_investigadores (
  proyecto_id INT,
  investigador_codigo TEXT,
  PRIMARY KEY (proyecto_id, investigador_codigo)
```

```
);
```

Segunda Forma Normal (2NF)

Cada columna no clave debe depender de la clave completa, no solo de parte de ella.

```
-- MAL: depende solo del proyecto, no de (proyecto, investigador)
CREATE TABLE proyecto_asignaciones (
    proyecto_id INT,
    investigador_codigo TEXT,
    proyecto_nombre TEXT, -- Depende solo de proyecto_id
    PRIMARY KEY (proyecto_id, investigador_codigo)
);

-- BIEN: separar en tablas
CREATE TABLE proyectos (
    id INT PRIMARY KEY,
    nombre TEXT
);

CREATE TABLE proyecto_asignaciones (
    proyecto_id INT REFERENCES proyectos(id),
    investigador_codigo TEXT REFERENCES personal(codigo),
    PRIMARY KEY (proyecto_id, investigador_codigo)
);
```

Tercera Forma Normal (3NF)

Las columnas no clave no deben depender de otras columnas no clave.

```
-- MAL: departamento_nombre depende de departamento_codigo
CREATE TABLE personal (
    codigo TEXT PRIMARY KEY,
    nombre TEXT,
    departamento_codigo TEXT,
    departamento_nombre TEXT
);

-- BIEN: tabla separada para departamentos
CREATE TABLE departamentos (
    codigo TEXT PRIMARY KEY,
    nombre TEXT
);

CREATE TABLE personal (
    codigo TEXT PRIMARY KEY,
    nombre TEXT,
    departamento_codigo TEXT REFERENCES departamentos(codigo)
);
```

Normalizando el ICB

Elisa encontró múltiples problemas en la base de datos del instituto:

1. **Tabla `personal`** tenía el nombre del departamento repetido para cada empleado.

2. **Tabla `accesos`** guardaba el nombre completo del empleado, no solo su código.

3. **Tabla `proyectos`** tenía una columna con IDs separados por comas.

—Un diseño así permite inconsistencias —explicó Elisa—. Y las inconsistencias son donde se esconden los fraudes.

Problemas comunes de diseño

Problema	Síntoma
Redundancia	Mismos datos en múltiples filas
Anomalía de actualización	Cambiar un valor requiere múltiples UPDATEs
Anomalía de inserción	No puedes crear un registro sin otro relacionado
Anomalía de borrado	Borrar un registro elimina datos no relacionados

La Revelación

Mientras normalizaba la base de datos mentalmente, Elisa se dio cuenta de algo. La tabla `transacciones\_ocultas` tenía una columna llamada `proyecto\_asociado`. Consultó:

```
SELECT DISTINCT proyecto_asociado FROM transacciones_ocultas;
```

Los proyectos eran: Cerbero, Ícaro, Fénix... y uno más: **Talos**.

—¿Qué es Talos? —preguntó Rovira.

Elisa buscó en la base de datos:

```
SELECT * FROM proyectos WHERE nombre = 'Talos';
```

No había registros. El proyecto no existía en la tabla oficial. Pero las transacciones sí lo mencionaban.

—Talos es el proyecto fantasma —dijo Elisa—. Un proyecto que no existe oficialmente pero que consume fondos. Vilaró creó un proyecto falso para desviar dinero.

—¿Y Héctor lo descubrió?

—Talos es el motivo del asesinato.

Enigma SQL #12: Diseñando la Verdad

1. ¿Qué anomalías de diseño encuentras en la tabla `accesos` si almacena `nombre\_completo` en lugar de solo `usuario\_codigo`? Propón una estructura normalizada.

2. Crea una consulta que detecte proyectos con transacciones pero sin registro en la tabla oficial `proyectos`.

3. Si tuvieras que rediseñar la base de datos del ICB desde cero, ¿qué tablas crearías y qué relaciones tendrían? Escribe el esquema SQL.

```
-- Tus consultas aquí
```

Capítulo 13: El Último Query (Parte 1)

—Proyecto Talos —dijo Elisa, proyectando el esquema en la pantalla—. No existe en los registros oficiales, pero consumió 2.3 millones de euros en dos años.

—¿Para qué? —preguntó el inspector jefe.

—No lo sé. Pero Héctor lo descubrió. Y por eso está muerto.

Elisa necesitaba reconstruir los datos completos de Talos. Las tablas estaban fragmentadas, los logs parcialmente borrados. Era como armar un rompecabezas con piezas faltantes.

Necesitaba una consulta que uniera todo: transacciones, accesos, personal, modificaciones. Pero también necesitaba cálculos complejos y datos ordenados jerárquicamente.

—Hora de usar CTEs —dijo Elisa.

Concepto SQL: CTEs (Common Table Expressions)

Un **CTE** es una tabla temporal definida dentro de una consulta. Es como una vista desechable que existe solo durante la ejecución.

CTE simple

```
WITH transferencias_sospechosas AS (  
    SELECT * FROM transacciones  
    WHERE monto > 100000  
        AND beneficiario NOT IN (  
            SELECT codigo FROM proveedores_autorizados  
        )  
)  
SELECT t.*, p.nombre as aprobador_nombre  
FROM transferencias_sospechosas t  
JOIN personal p ON t.aprobador = p.codigo;
```

Múltiples CTEs

```
WITH  
    accesos_nocturnos AS (  
        SELECT * FROM accesos  
        WHERE strftime('%H', fecha) BETWEEN '22' AND '06'  
    ),  
    transacciones_recientes AS (  
        SELECT * FROM transacciones  
        WHERE fecha > '2026-01-01'
```

```

)
SELECT a.usuario, a.fecha, t.id as transaccion_id, t.monto
FROM accesos_nocturnos a
JOIN transacciones_recientes t ON a.usuario = t.usuario_creacion
    AND date(a.fecha) = date(t.fecha)
ORDER BY a.fecha DESC;

```

CTE con agregaciones

```

WITH
estadisticas_usuario AS (
    SELECT usuario,
           COUNT(*) as total_accesos,
           COUNT(DISTINCT date(fecha)) as dias_activo
    FROM accesos
    GROUP BY usuario
)
SELECT p.nombre, e.total_accesos, e.dias_activo,
       ROUND(CAST(e.total_accesos AS REAL) / e.dias_activo, 1) as accesos_por_dia
FROM estadisticas_usuario e
JOIN personal p ON e.usuario = p.codigo
ORDER BY accesos_por_dia DESC;

```

CTEs recursivos

Los CTEs recursivos se usan para datos jerárquicos (organigramas, árboles):

```

-- Organigrama del ICB
WITH RECURSIVE organigrama AS (
    -- Caso base: el director
    SELECT codigo, nombre, jefe_codigo, 0 as nivel
    FROM personal
    WHERE jefe_codigo IS NULL

    UNION ALL

    -- Paso recursivo: subordinados
    SELECT p.codigo, p.nombre, p.jefe_codigo, o.nivel + 1
    FROM personal p
    JOIN organigrama o ON p.jefe_codigo = o.codigo
)
SELECT * FROM organigrama ORDER BY nivel, nombre;

```

La Consulta Final

Elisa construyó un CTE complejo para reunir todas las piezas:

```

WITH
accesos_sospechosos AS (
    SELECT usuario, fecha, ip_origen
    FROM accesos
    WHERE laboratorio = 7

```

```

        AND fecha > '2026-06-01'
    ),
    transacciones_sospechosas AS (
        SELECT id, monto, fecha, usuario_creacion,
               beneficiario, proyecto_asociado
        FROM transacciones_ocultas
        WHERE monto > 50000
    ),
    modificaciones_sospechosas AS (
        SELECT usuario, tabla, fecha_modificacion, ip_origen
        FROM logs_modificaciones
        WHERE tabla IN ('transacciones', 'inventario_quimico', 'cerbero_config')
               AND fecha_modificacion > '2026-06-01'
    ),
    linea_tiempo AS (
        SELECT fecha, 'acceso' as evento, usuario, NULL as detalle
        FROM accesos_sospechosos
        UNION ALL
        SELECT fecha, 'transaccion' as evento, usuario_creacion,
               CAST(monto AS TEXT) as detalle
        FROM transacciones_sospechosas
        UNION ALL
        SELECT fecha_modificacion as fecha, 'modificacion' as evento,
               usuario, tabla as detalle
        FROM modificaciones_sospechosas
    )
    SELECT fecha, evento, usuario, detalle
    FROM linea_tiempo
    ORDER BY fecha;

```

El resultado mostraba una coreografía perfecta: accesos, transacciones y modificaciones se sucedían en secuencia, como un ballet planeado. Y en el centro de todo, dos nombres: Marta Robles y Albert Vilaró.

Pero faltaba una pieza. Entre las transacciones de Talos, una se destacaba:

Monto	Fecha	Beneficiario
750,000 EUR	2026-07-14 20:30	SwissCredit AG

—Esa transferencia se ejecutó antes del asesinato —dijo Elisa—. Y el beneficiario no es el ICB. Es una cuenta personal.

—¿De quién? —preguntó Rovira.

—Aún no lo sé. Pero voy a descubrirlo.

Enigma SQL #13: Reconstruyendo la Verdad

1. Usando CTEs, crea una consulta que liste todos los eventos del 14 de julio de 2026 ordenados cronológicamente: accesos, transacciones y modificaciones.
2. Con un CTE recursivo, construye el organigrama del ICB y encuentra a qué nivel está Vilaró respecto al director.
3. ¿Cuánto dinero total se transfirió al proyecto Talos? Usa un CTE para calcularlo.

-- Tus consultas aquí

Capítulo 14: El Último Query (Parte 2)

Faltaba la pieza final: ¿quién era el beneficiario real de la cuenta SwissCredit? La base de datos no tenía esa información, pero los metadatos de las transacciones sí contenían pistas.

Elisa observó el patrón de las transacciones. Cada transferencia a SwissCredit estaba precedida por una serie de transacciones más pequeñas. Como si alguien estuviera consolidando fondos.

Era un patrón conocido en lavado de dinero: muchas entradas pequeñas, una salida grande. Necesitaba una herramienta analítica más potente para detectar el patrón.

—Window functions —dijo en voz alta—. Es lo único que puede revelar este patrón.

Concepto SQL: Window Functions

Las **window functions** realizan cálculos en un conjunto de filas relacionadas con la fila actual. A diferencia de GROUP BY, no colapsan filas.

Sintaxis

```
SELECT columna,
       funcion_ventana() OVER (
         PARTITION BY columna_grupo
         ORDER BY columna_orden
         frame_clause
       ) as resultado
FROM tabla;
```

ROW_NUMBER, RANK, DENSE_RANK

```
-- Numerar transacciones por fecha
SELECT id, monto, fecha,
       ROW_NUMBER() OVER (ORDER BY fecha) as num_fila,
       RANK() OVER (ORDER BY monto DESC) as ranking_monto,
       DENSE_RANK() OVER (ORDER BY monto DESC) as ranking_sin_huecos
FROM transacciones;
```

LAG y LEAD

Acceder a filas anteriores o siguientes:

```
-- Comparar cada transacción con la anterior del mismo usuario
SELECT usuario, fecha, monto,
       LAG(monto) OVER (PARTITION BY usuario ORDER BY fecha) as monto_anterior,
```

```

        monto - LAG(monto) OVER (PARTITION BY usuario ORDER BY fecha) as diferencia,
        LEAD(fecha) OVER (PARTITION BY usuario ORDER BY fecha) as siguiente_transaccion
FROM transacciones
ORDER BY usuario, fecha;

```

SUM, AVG con ventanas

```

-- Acumulado por usuario
SELECT usuario, fecha, monto,
       SUM(monto) OVER (PARTITION BY usuario ORDER BY fecha) as acumulado
FROM transacciones;

-- Promedio móvil de 3 transacciones
SELECT fecha, monto,
       AVG(monto) OVER (ORDER BY fecha ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) as media_movil
FROM transacciones;

```

FIRST_VALUE y LAST_VALUE

```

-- Primera y última transacción de cada usuario
SELECT usuario, fecha, monto,
       FIRST_VALUE(monto) OVER (PARTITION BY usuario ORDER BY fecha) as primer_monto,
       LAST_VALUE(monto) OVER (PARTITION BY usuario ORDER BY fecha
                               ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as ultimo_monto
FROM transacciones;

```

Detección de patrones

```

-- Detectar transacciones que son > 2x la anterior (pico sospechoso)
WITH con_anterior AS (
    SELECT fecha, monto,
           LAG(monto) OVER (ORDER BY fecha) as monto_anterior
    FROM transacciones
)
SELECT fecha, monto, monto_anterior,
       ROUND(CAST(monto AS REAL) / monto_anterior, 2) as ratio
FROM con_anterior
WHERE monto_anterior IS NOT NULL
      AND CAST(monto AS REAL) / monto_anterior > 2
ORDER BY ratio DESC;

```

El Patrón Revelado

Elisa aplicó window functions a las transacciones:

```

WITH transacciones_ordenadas AS (
    SELECT id, fecha, monto, beneficiario, usuario_creacion,
           LAG(fecha) OVER (PARTITION BY usuario_creacion ORDER BY fecha) as fecha_anterior,
           LAG(monto) OVER (PARTITION BY usuario_creacion ORDER BY fecha) as monto_anterior,
           SUM(monto) OVER (PARTITION BY usuario_creacion ORDER BY fecha) as acumulado
    FROM transacciones_ocultas
)

```

```

WHERE proyecto_asociado = 'Talos'
)
SELECT *,
CASE
    WHEN monto_anterior IS NOT NULL
    AND CAST(monto AS REAL) / monto_anterior > 5
    THEN '*** PICO SOSPECHOSO ***'
    ELSE ''
END as alerta
FROM transacciones_ordenadas
WHERE beneficiario = 'SwissCredit AG'
ORDER BY fecha;

```

El resultado mostró que las transferencias grandes a SwissCredit siempre iban precedidas de 5-7 transacciones pequeñas. El patrón era claro: acumulación y consolidación.

Pero había un detalle más: la última transacción a SwissCredit — 750,000 EUR — no tenía transacciones pequeñas precedentes. Era la única que rompía el patrón.

—Esto no es lavado de dinero —dijo Elisa—. Es un pago. Un pago único.

—¿Un soborno? —preguntó Rovira.

—O el pago por un asesinato.

Enigma SQL #14: Siguiendo el Dinero

1. Usando LAG, encuentra las transacciones donde el monto es al menos 3 veces mayor que la transacción anterior del mismo usuario.
2. Calcula el acumulado de transacciones por beneficiario, ordenado por fecha. ¿Cuánto recibió SwissCredit en total?
3. Usando window functions, asigna un número de orden a cada transacción dentro de cada día. ¿Cuántas transacciones hubo el 14 de julio?

```
-- Tus consultas aquí
```

Capítulo 15: El Epílogo

La Verdad

—Héctor no solo descubrió Talos —dijo Elisa, de pie frente al tribunal—. Descubrió que el proyecto financiaba operaciones encubiertas del ICB. Operaciones que el gobierno había autorizado pero que nunca aparecerían en los presupuestos oficiales.

Vilaró la miraba desde el banquillo. A su lado, Marta Robles mantenía la cabeza baja.

—Pero entonces —intervino el fiscal—, ¿por qué mataron al doctor Blasco?

—Porque Héctor descubrió que una parte del dinero no iba a operaciones encubiertas. Iba a una cuenta privada. Vilaró estaba robando, usando el proyecto Talos como pantalla.

—Y cuando Blasco lo descubrió...

—Vilaró lo silenció. Pero cometió un error: usó SQL para ocultar su rastro. Y SQL, bien usado, siempre revela la verdad.

Resumen SQL

A lo largo de esta investigación, has aprendido:

Nivel Básico

- **SELECT y FROM:** Leer datos de tablas
- **WHERE:** Filtrar filas con condiciones
- **ORDER BY:** Ordenar resultados
- **LIMIT:** Limitar resultados
- **LIKE:** Búsqueda con patrones
- **IN, BETWEEN:** Rangos y listas

Nivel Intermedio

- **JOIN (INNER, LEFT, RIGHT, FULL):** Combinar tablas
- **GROUP BY y funciones de agregación:** Resumir datos (COUNT, SUM, AVG, MAX, MIN)
- **HAVING:** Filtrar grupos
- **Funciones de fecha:** DATE, strftime, datetime
- **Subconsultas:** Consultas dentro de consultas

- **Funciones de cadena:** SUBSTR, UPPER, LOWER, REPLACE, TRIM
- **CASE:** Condicionales en SQL

Nivel Avanzado

- **Índices:** Optimizar consultas
 - **Vistas:** Simplificar consultas complejas
 - **Transacciones:** Controlar cambios (BEGIN, COMMIT, ROLLBACK)
 - **Triggers:** Automatizar acciones en la base de datos
 - **Normalización:** Diseñar bases de datos eficientes
 - **CTEs:** Consultas temporales (WITH)
 - **CTEs Recursivos:** Datos jerárquicos
 - **Window Functions:** ROW_NUMBER, LAG, LEAD, SUM/AVG con ventanas
-

El Juicio

El trigger de Héctor, combinado con las window functions que Elisa aplicó, demostró que Vilaró había usado la cuenta de Blasco para ejecutar la transferencia de 750,000 EUR. La toxina T-47 fue enviada al laboratorio por orden de Vilaró, quien luego borró el registro del inventario.

Marta Robles, enfrentando cargos reducidos por cooperar, testificó que Vilaró la había amenazado para que participara en el encubrimiento.

Albert Vilaró fue condenado a 25 años de prisión por asesinato, malversación de fondos y obstrucción a la justicia.

Después

Elisa Riera fue promovida a jefa del departamento de análisis forense de datos del ICB. Su primera decisión fue rediseñar la base de datos del instituto.

—Nunca más —dijo— un diseño defectuoso ocultará un crimen. Los datos deben ser claros, consistentes y auditables. Como la verdad.

En su escritorio, una placa recordaba a su mentor:

> *"SELECT la verdad FROM los datos WHERE la justicia = true;"*

—Héctor —murmuró—. Tu último query se ejecutó.

Enigma SQL #15: El Caso Cerrado

1. Usando todo lo aprendido, escribe una consulta que resuma el caso: total de dinero desviado, número de transacciones, implicados y periodo de tiempo.

2. Crea una vista ``v_resumen_caso_blasco`` que pueda usar un fiscal para presentar la evidencia.

3. Si fueras Elisa, ¿qué índice crearías para prevenir futuros fraudes en la tabla ``transacciones``?

-- Tus consultas aquí

Conclusión

Has llegado al final de **SQL Letal: El Misterio de la Base de Datos**.

Pero tu viaje con SQL no termina aquí.

SQL es el lenguaje de los datos. Está en todas partes: en tu banco, en tu hospital, en tu red social favorita, en el sistema que usas para escuchar música o pedir comida. Cada vez que interactúas con una aplicación moderna, hay una base de datos trabajando detrás.

¿Qué sigue?

- **Practica:** Los ejercicios de este libro están diseñados para SQLite, el motor de bases de datos más accesible. Instálalo y prueba las consultas.
- **Explora:** PostgreSQL y MySQL son los motores más usados en producción. Aprende sus diferencias.
- **Construye:** Crea tu propia base de datos. Un inventario personal, un sistema de seguimiento de lecturas, lo que sea.
- **Profundiza:** Los conceptos que viste aquí son la punta del iceberg. Explora stored procedures, funciones ventana avanzadas, JSON en SQL, y más.

Recursos

- **SQLite:** <https://sqlite.org>
- **PostgreSQL:** <https://postgresql.org>
- **DB Browser for SQLite:** <https://sqlitebrowser.org> (interfaz gráfica)
- **SQLZoo:** <https://sqlzoo.net> (ejercicios interactivos)
- **LeetCode SQL:** <https://leetcode.com/problemset/database/> (desafíos)

Palabras finales

La próxima vez que alguien te diga que los datos no mienten, recuerda: los datos no mienten, pero las personas sí. Y SQL es la herramienta que necesitas para descubrir la verdad.

Como decía el Dr. Blasco:

> *"En los datos se esconde la verdad. En SQL, la respuesta."*

Sigue consultando. Sigue descubriendo.

— Alex Goyzueta Delgado

Barcelona, 2026

Apéndice: Soluciones a los Enigmas

Enigma 1: Los Registros Perdidos

```
-- 1. ¿Cuántos experimentos hay registrados?
SELECT COUNT(*) FROM experimentos;

-- 2. Muestra todos ordenados por fecha descendente
SELECT * FROM experimentos ORDER BY fecha DESC;

-- 3. ¿Qué columnas tiene la tabla?
-- id, nombre, fecha, investigador, resultado, proyecto
-- Puedes obtenerlas con:
PRAGMA table_info(experimentos);
```

Enigma 2: El Rastro del Profesor

```
-- 1. Accesos nocturnos del Dr. Blasco
SELECT * FROM accesos
WHERE usuario = 'hblasco'
      AND strftime('%H', fecha) >= '22';

-- 2. Experimentos con resultado NULL
SELECT COUNT(*) FROM experimentos
WHERE resultado IS NULL;

-- 3. Proyectos con "Cerbero"
SELECT * FROM experimentos
WHERE proyecto LIKE '%Cerbero%';
```

Enigma 3: ¿Quién más sabía?

```
-- 1. Investigadores con acceso a Cerbero
SELECT DISTINCT p.nombre, p.codigo
FROM personal p
JOIN proyecto_investigadores pi ON p.codigo = pi.investigador_codigo
WHERE pi.proyecto_id = 'Cerbero';

-- 2. Accesos al laboratorio 7 en junio
SELECT a.fecha, a.usuario, p.nombre, a.puerta
FROM accesos a
JOIN personal p ON a.usuario = p.codigo
WHERE a.laboratorio = 7
```

```

AND a.fecha BETWEEN '2026-06-01' AND '2026-06-30';

-- 3. Accedieron pero no son investigadores de Cerbero
SELECT DISTINCT a.usuario
FROM accesos a
WHERE a.laboratorio = 7
AND a.usuario NOT IN (
    SELECT investigador_codigo
    FROM proyecto_investigadores
    WHERE proyecto_id = 'Cerbero'
);

```

Enigma 4: La Agenda Oculta

```

-- 1. Accesos por empleado en junio (orden descendente)
SELECT a.usuario, p.nombre, COUNT(*) as total
FROM accesos a
JOIN personal p ON a.usuario = p.codigo
WHERE a.fecha BETWEEN '2026-06-01' AND '2026-06-30'
GROUP BY a.usuario
ORDER BY total DESC;

-- 2. Días con más de 5 accesos fallidos
SELECT DATE(fecha) as dia, COUNT(*) as fallos
FROM accesos
WHERE acceso_concedido = 0
GROUP BY DATE(fecha)
HAVING COUNT(*) > 5;

-- 3. Promedio de accesos por hora
SELECT strftime('%H', fecha) as hora, COUNT(*) as total
FROM accesos
GROUP BY strftime('%H', fecha)
ORDER BY hora;

```

Enigma 5: La Línea de Tiempo

```

-- 1. Modificaciones del 15 de julio
SELECT * FROM logs_modificaciones
WHERE DATE(fecha_modificacion) = '2026-07-15'
ORDER BY fecha_modificacion;

-- 2. Modificaciones nocturnas (22:00-06:00)
SELECT * FROM logs_modificaciones
WHERE fecha_modificacion BETWEEN '2026-07-14 22:00' AND '2026-07-15 06:00';

-- 3. Diferencia entre última modif de hblasco y primera de mroble
SELECT
    (SELECT MAX(fecha_modificacion) FROM logs_modificaciones WHERE usuario = 'hblasco') as ultima_hblasco,
    (SELECT MIN(fecha_modificacion) FROM logs_modificaciones WHERE usuario = 'mroble') as primera_mroble,
    ROUND(julianday(
        (SELECT MIN(fecha_modificacion) FROM logs_modificaciones WHERE usuario = 'mroble')

```

```

) - julianday(
    (SELECT MAX(fecha_modificacion) FROM logs_modificaciones WHERE usuario = 'hblasco')
) * 24 * 60, 2) as minutos_diferencia;

```

Enigma 6: La Red Oculta

```

-- 1. Conexiones externas después del crimen
SELECT * FROM conexiones
WHERE ip_origen NOT LIKE '10.0.%'
    AND fecha > '2026-07-15 05:30';

-- 2. IPs usadas por Marta
SELECT DISTINCT ip_origen FROM conexiones
WHERE usuario = 'mroble'
    AND fecha > DATE('now', '-30 days');

-- 3. Usuarios con accesos nocturnos sobre el promedio
WITH nocturnos AS (
    SELECT usuario, COUNT(*) as accesos_nocturnos
    FROM accesos
    WHERE strftime('%H', fecha) BETWEEN '22' AND '06'
    GROUP BY usuario
)
SELECT n.* FROM nocturnos n
WHERE n.accesos_nocturnos > (
    SELECT AVG(accesos_nocturnos) FROM nocturnos
);

```

Enigma 7: El Rastro del Dinero

```

-- 1. Transacciones por año
SELECT strftime('%Y', fecha) as anio, COUNT(*) as total, SUM(monto) as suma
FROM transacciones_ocultas
GROUP BY strftime('%Y', fecha)
ORDER BY anio;

-- 2. Crear índice y verificar
CREATE INDEX idx_beneficiario_ocultas ON transacciones_ocultas(beneficiario);
EXPLAIN QUERY PLAN
SELECT * FROM transacciones_ocultas WHERE beneficiario = 'ICB';

-- 3. Transacción de mayor monto
SELECT * FROM transacciones_ocultas
ORDER BY monto DESC
LIMIT 1;

```

Enigma 8: La Investigación Simplificada

```

-- 1. Vista de accesos al laboratorio 7 de Sistemas
CREATE VIEW v_accesos_lab7_sistemas AS
SELECT a.*, p.nombre, p.departamento
FROM accesos a

```

```

JOIN personal p ON a.usuario = p.codigo
WHERE a.laboratorio = 7
      AND p.departamento = 'Sistemas'
      AND a.fecha BETWEEN '2026-06-01' AND '2026-07-31';

-- 2. Empleados que accedieron a lab7 y almacén el mismo día
SELECT a.usuario, DATE(a.fecha) as dia
FROM accesos a
WHERE a.laboratorio = 7
INTERSECT
SELECT a2.usuario, DATE(a2.fecha)
FROM accesos a2
WHERE a2.laboratorio = 'AlmacenQCA';

-- 3. Accesos por departamento al lab7
CREATE VIEW v_accesos_por_depto AS
SELECT p.departamento, COUNT(*) as total_accesos
FROM accesos a
JOIN personal p ON a.usuario = p.codigo
WHERE a.laboratorio = 7
GROUP BY p.departamento;

```

Enigma 9: La Cadena de Transacciones

```

-- 1. Transferencia de 500,000 EUR
BEGIN TRANSACTION;
SELECT saldo FROM cuentas WHERE id = 1;
-- Asumiendo saldo suficiente
UPDATE cuentas SET saldo = saldo - 500000 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 500000 WHERE id = 2;
INSERT INTO registro_transferencias (origen, destino, monto, fecha)
VALUES (1, 2, 500000, datetime('now'));
COMMIT;

-- 2. Transacciones con confirmación < 5 min
SELECT id, fecha_inicio, fecha_confirmacion,
       ROUND((julianday(fecha_confirmacion) - julianday(fecha_inicio)) * 24 * 60, 1) as minutos
FROM transacciones
WHERE fecha_confirmacion IS NOT NULL
      AND (julianday(fecha_confirmacion) - julianday(fecha_inicio)) * 24 * 60 < 5;

-- 3. Transacciones con cuenta de fallecido
SELECT t.* FROM transacciones t
JOIN personal p ON t.usuario_ejecutor = p.codigo
WHERE t.fecha > p.fecha_muerte;

```

Enigma 10: Decodificando el Mensaje

```

-- 1. Primeros 4 caracteres de concepto
SELECT id, monto, SUBSTR(concepto, 1, 4) as prefijo,
       COUNT(*) OVER (PARTITION BY SUBSTR(concepto, 1, 4)) as frecuencia
FROM transacciones_ocultas;

```

```

-- 2. Clasificación por monto
SELECT monto,
       CASE
         WHEN monto < 1000 THEN 'Bajo'
         WHEN monto BETWEEN 1000 AND 50000 THEN 'Medio'
         ELSE 'Alto'
       END as categoria,
       COUNT(*) as cantidad
FROM transacciones_ocultas
GROUP BY categoria;

-- 3. Nombre completo y total de transacciones
SELECT p.nombre || ' ' || p.apellido as nombre_completo,
       COUNT(t.id) as total_transacciones
FROM personal p
LEFT JOIN transacciones_ocultas t ON p.codigo = t.usuario_creacion
GROUP BY p.codigo;

```

Enigma 11: El Guardián Silencioso

```

-- 1. Trigger que registra eliminaciones
CREATE TABLE IF NOT EXISTS log_eliminaciones (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  tabla TEXT,
  registro_id INTEGER,
  usuario TEXT,
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER trg_log_eliminaciones
AFTER DELETE ON transacciones
FOR EACH ROW
BEGIN
  INSERT INTO log_eliminaciones (tabla, registro_id, usuario)
  VALUES ('transacciones', OLD.id,
          (SELECT usuario FROM conexiones ORDER BY fecha DESC LIMIT 1));
END;

-- 2. Auditoría del 14 de julio
SELECT * FROM auditoria_oculta
WHERE DATE(fecha) = '2026-07-14';

-- 3. Trigger que impide modificar monto de transacciones confirmadas
CREATE TRIGGER trg_proteger_monto
BEFORE UPDATE OF monto ON transacciones
FOR EACH ROW
BEGIN
  SELECT CASE
    WHEN OLD.estado = 'CONFIRMADA'
    THEN RAISE(ABORT, 'No se puede modificar el monto de una transacción confirmada')
  END;
END;

```

```
END;
```

Enigma 12: Diseñando la Verdad

```
-- 1. Anomalías en tabla accesos
-- Si acceso tiene nombre_completo:
-- - Redundancia (el nombre se repite en cada acceso)
-- - Anomalía de actualización (si alguien cambia de nombre, hay que actualizar N filas)

-- Estructura normalizada:
CREATE TABLE accesos_normalizado (
    id INTEGER PRIMARY KEY,
    usuario_codigo TEXT REFERENCES personal(codigo),
    fecha TIMESTAMP,
    puerta TEXT,
    laboratorio TEXT,
    acceso_concedido BOOLEAN
);

-- 2. Proyectos con transacciones pero sin registro oficial
SELECT DISTINCT t.proyecto_asociado
FROM transacciones_ocultas t
WHERE t.proyecto_asociado NOT IN (
    SELECT nombre FROM proyectos
);

-- 3. Esquema óptimo
CREATE TABLE departamentos (
    codigo TEXT PRIMARY KEY,
    nombre TEXT NOT NULL
);

CREATE TABLE personal (
    codigo TEXT PRIMARY KEY,
    nombre TEXT NOT NULL,
    apellido TEXT,
    email TEXT,
    cargo TEXT,
    departamento_codigo TEXT REFERENCES departamentos(codigo),
    jefe_codigo TEXT REFERENCES personal(codigo)
);

CREATE TABLE proyectos (
    id TEXT PRIMARY KEY,
    nombre TEXT NOT NULL,
    descripcion TEXT,
    presupuesto REAL,
    fecha_inicio DATE,
    responsable_codigo TEXT REFERENCES personal(codigo)
);

CREATE TABLE proyecto_investigadores (
    proyecto_id TEXT REFERENCES proyectos(id),
    investigador_codigo TEXT REFERENCES personal(codigo),
    PRIMARY KEY (proyecto_id, investigador_codigo)
```

```

);
CREATE TABLE accesos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    usuario_codigo TEXT REFERENCES personal(codigo),
    fecha TIMESTAMP,
    puerta TEXT,
    laboratorio TEXT,
    acceso_concedido BOOLEAN
);
CREATE TABLE transacciones (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    monto REAL NOT NULL,
    fecha TIMESTAMP,
    concepto TEXT,
    beneficiario TEXT,
    proyecto_id TEXT REFERENCES proyectos(id),
    usuario_creacion TEXT REFERENCES personal(codigo),
    estado TEXT DEFAULT 'PENDIENTE'
);

```

Enigma 13: Reconstruyendo la Verdad

```

-- 1. Eventos del 14 de julio
WITH eventos AS (
    SELECT fecha, 'acceso' as tipo, usuario, CAST(puerta AS TEXT) as detalle
    FROM accesos
    WHERE DATE(fecha) = '2026-07-14'
    UNION ALL
    SELECT fecha, 'transaccion', usuario_creacion, CAST(monto AS TEXT)
    FROM transacciones_ocultas
    WHERE DATE(fecha) = '2026-07-14'
    UNION ALL
    SELECT fecha_modificacion, 'modificacion', usuario, tabla
    FROM logs_modificaciones
    WHERE DATE(fecha_modificacion) = '2026-07-14'
)
SELECT * FROM eventos ORDER BY fecha;

-- 2. Organigrama recursivo
WITH RECURSIVE organigrama AS (
    SELECT codigo, nombre, jefe_codigo, 0 as nivel
    FROM personal
    WHERE jefe_codigo IS NULL
    UNION ALL
    SELECT p.codigo, p.nombre, p.jefe_codigo, o.nivel + 1
    FROM personal p
    JOIN organigrama o ON p.jefe_codigo = o.codigo
)
SELECT * FROM organigrama WHERE nombre LIKE '%Vilaró%' OR nombre LIKE '%Director%';

-- 3. Total transferido a Talos
WITH talos_transacciones AS (

```

```

SELECT * FROM transacciones_ocultas
WHERE proyecto_asociado = 'Talos'
)
SELECT COUNT(*) as num_transacciones, SUM(monto) as total
FROM talos_transacciones;

```

Enigma 14: Siguiendo el Dinero

```

-- 1. Transacciones con monto >= 3x la anterior (mismo usuario)
WITH con_lag AS (
    SELECT id, usuario_creacion, fecha, monto,
           LAG(monto) OVER (PARTITION BY usuario_creacion ORDER BY fecha) as monto_anterior
    FROM transacciones_ocultas
)
SELECT * FROM con_lag
WHERE monto_anterior IS NOT NULL
    AND CAST(monto AS REAL) / monto_anterior >= 3;

-- 2. Acumulado por beneficiario
SELECT beneficiario, fecha, monto,
       SUM(monto) OVER (PARTITION BY beneficiario ORDER BY fecha) as acumulado
FROM transacciones_ocultas
ORDER BY beneficiario, fecha;

-- 3. Transacciones por orden dentro de cada día
SELECT id, fecha, monto,
       ROW_NUMBER() OVER (PARTITION BY DATE(fecha) ORDER BY fecha) as orden_dia
FROM transacciones_ocultas
WHERE DATE(fecha) = '2026-07-14';

```

Enigma 15: El Caso Cerrado

```

-- 1. Resumen del caso
SELECT
    'Talos' as proyecto,
    COUNT(*) as total_transacciones,
    SUM(monto) as total_desviado,
    COUNT(DISTINCT usuario_creacion) as implicados,
    MIN(fecha) as primer_transaccion,
    MAX(fecha) as ultima_transaccion
FROM transacciones_ocultas
WHERE proyecto_asociado = 'Talos';

-- 2. Vista para el fiscal
CREATE VIEW v_resumen_caso_blasco AS
SELECT
    t.id as transaccion_id,
    t.fecha,
    t.monto,
    t.beneficiario,
    t.proyecto_asociado,
    p.nombre as creador_nombre,

```

```

p.departamento,
CASE
    WHEN t.beneficiario NOT IN (SELECT codigo FROM proveedores_autorizados)
    THEN 'NO AUTORIZADO'
    ELSE 'AUTORIZADO'
END as estado_beneficiario
FROM transacciones_ocultas t
JOIN personal p ON t.usuario_creacion = p.codigo
WHERE t.proyecto_asociado = 'Talos';

-- Uso: SELECT * FROM v_resumen_caso_blasco;

-- 3. Índices preventivos
CREATE INDEX idx_transacciones_fecha ON transacciones(fecha);
CREATE INDEX idx_transacciones_beneficiario ON transacciones(beneficiario);
CREATE INDEX idx_transacciones_usuario ON transacciones(usuario_creacion);
CREATE INDEX idx_transacciones_proyecto ON transacciones(proyecto_asociado);
-- Índice compuesto para consultas frecuentes de investigación
CREATE INDEX idx_transacciones_investigacion
ON transacciones(fecha, usuario_creacion, beneficiario, monto);

```

Sobre el Autor

Alex Goyzueta Delgado es analista de datos, programador y escritor peruano residente en Barcelona.

Combina su experiencia en tecnología con una pasión por la narrativa para crear libros que enseñan programación y ciencia de datos a través de historias de misterio.

Es autor de:

- **Código Asesino: El Misterio de la Clave Oculta** — Python + Misterio Inca
- **Raíces Nuevas** — Guía para combatir la ansiedad del inmigrante
- **Estadística Mortal: El Enigma del Conjunto Vacío** — Estadística + Misterio
- **SQL Letal: El Misterio de la Base de Datos** — SQL + Misterio Forense

Cree firmemente que la mejor manera de aprender tecnología es a través de historias que emocionen y desafíen.

Contacto: alexgoyzueta2018@gmail.com

Repositorio del proyecto:

[github.com/SistGoyPeru/Libros](<https://github.com/SistGoyPeru/Libros>)